

# Paradigma de programación funcional en el backend

## Conclusión

El backend **utiliza elementos de programación funcional**, pero su paradigma principal es **imperativo / orientado a objetos**. La programación funcional se usa como una herramienta puntual (Streams, lambdas, métodos de referencia, `Optional`), dentro de una arquitectura clásica por capas: Controller → Service → Repository.

No es un backend funcional puro ni usa frameworks funcionales como Vavr, Arrow o functional interfaces avanzadas. Es un proyecto Spring Boot convencional con toques funcionales donde Java lo facilita.

---

## Evidencia de programación funcional en el código

### 1. Streams y Collectors (`ProductServiceImpl.java`)

```
return productRepository.search(normalizedCategory, normalizedName, sort)
    .stream()
    .map(this::toResponseDTO) // método de referencia (method reference)
    .collect(Collectors.toList());
```

- `stream()` → convierte la colección en un pipeline funcional.
- `.map(this::toResponseDTO)` → función de transformación aplicada a cada elemento (funciones de primera clase).
- `.collect(Collectors.toList())` → operación terminal que agrega los resultados.

### 2. `Optional` y lambdas (`ProductServiceImpl.java`)

```
private Product findEntityOrThrow(Long id) {
    return productRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("No se encontró el producto con id " + id));
}
```

- `Optional<T>` es un contenedor funcional (monádico) de Java.
- `orElseThrow` recibe un `Supplier` (lambda) que se ejecuta solo si el valor está ausente.
- Este patrón evita `if (product == null)` y encadena comportamiento de forma declarativa.

### 3. Inmutabilidad y colecciones inmutables

```
private static final Set<String> SORTABLE_FIELDS = Set.of("name", "category", "price", "stock", "id");
```

- `Set.of(...)` crea colecciones **inmutables**, un principio central de la programación funcional.
- El `record` (si se usara) o DTOs inmutables con Lombok `@Builder` también siguen esta filosofía.

### 4. Expresiones lambda en otros contextos

- El controlador usa funciones de orden superior indirectamente: `productService.getAllProducts(...)` delega la lógica.
- Spring Data JPA usa **queries declarativas** (JPQL) que comparten el

```
public ProductResponseDTO updateProduct(Long id, ProductRequestDTO request) {
    Product product = findEntityOrThrow(id);

    product.setName(request.getName().trim()); // mutación de estado
    product.setDescription(request.getDescription()); // mutación de estado
    product.setCategory(request.getCategory().trim()); // mutación de estado
    product.setPrice(request.getPrice()); // mutación de estado
    product.setStock(request.getStock()); // mutación de estado
    ...
}
```

- Las entidades JPA son **mutables** (setters).
- El estado se modifica **in-place**, algo prohibido en programación funcional pura.

## 2. Dependencia de estado externo (base de datos)

- `productRepository.save(product)` tiene efectos secundarios en la base de datos.
- `@Transactional` mantiene estado de sesión entre llamadas.
- Las funciones no son puras (no siempre devuelven el mismo resultado para los mismos argumentos).

## 3. @Service y @Repository son clases con estado

- Spring maneja beans con estado (singletons).
- La inyección de dependencias es un patrón orientado a objetos.

## 4. Controladores imperativos

```
@GetMapping("/{id}")
public ResponseEntity<ProductResponseDTO> getProductById(@PathVariable Long id) {
    return ResponseEntity.ok(productService.getProductById(id));
}
```

- Los endpoints son funciones imperativas secuenciales.
- No hay composición de funciones ni monadas reactivas (`Mono/Flux`).

## Resumen comparativo

| Característica                            | ¿Usada en el backend? |
|-------------------------------------------|-----------------------|
| Streams (stream, map, collect)            | Sí                    |
| Lambdas y métodos de referencia           | Sí                    |
| Optional (patrón monádico)                | Sí                    |
| Colecciones inmutables (Set.of, List.of)  | Sí                    |
| Funciones puras (sin efectos secundarios) | No                    |
| Inmutabilidad total de datos              | No                    |
| Composición de funciones avanzada         | No                    |
| Monadas reactivas (Mono y Flux)           | No                    |
| Framework funcional (Vavr, Arrow)         | No                    |

## Conclusión final

El backend **aplica programación**